PAYROLL MANAGEMENT SYSTEM

1. Title Page:
   Course Name: Advanced Programming (OOP)(PCCCS405)
   Lab Project Title: Multi-Tiered Employee Payroll Manager
   Student Name: Sagnik Ghosh
   Roll Number: 62
   Enrollment Number: 12024052019049
   Instructor Name: Soumadip Biswas
   Semester: Spring 2026

---

2. Problem Statement:
   Manual payroll processing is a practical problem prone to errors in fiscal logic and data management. Organizations often handle diverse employee types (Full-Time, Part-Time) with different salary structures, making manual tracking inefficient.
   The objective is to develop a console-based Multi-Tiered Employee Payroll Manager in Java. This system provides a simple interface to create, view, and store employee records. It utilizes an N-tier architecture (Model-View-Controller) to ensure separation of concerns and maintainability. Data persistence is achieved through file handling, ensuring records remain accessible after program restarts.

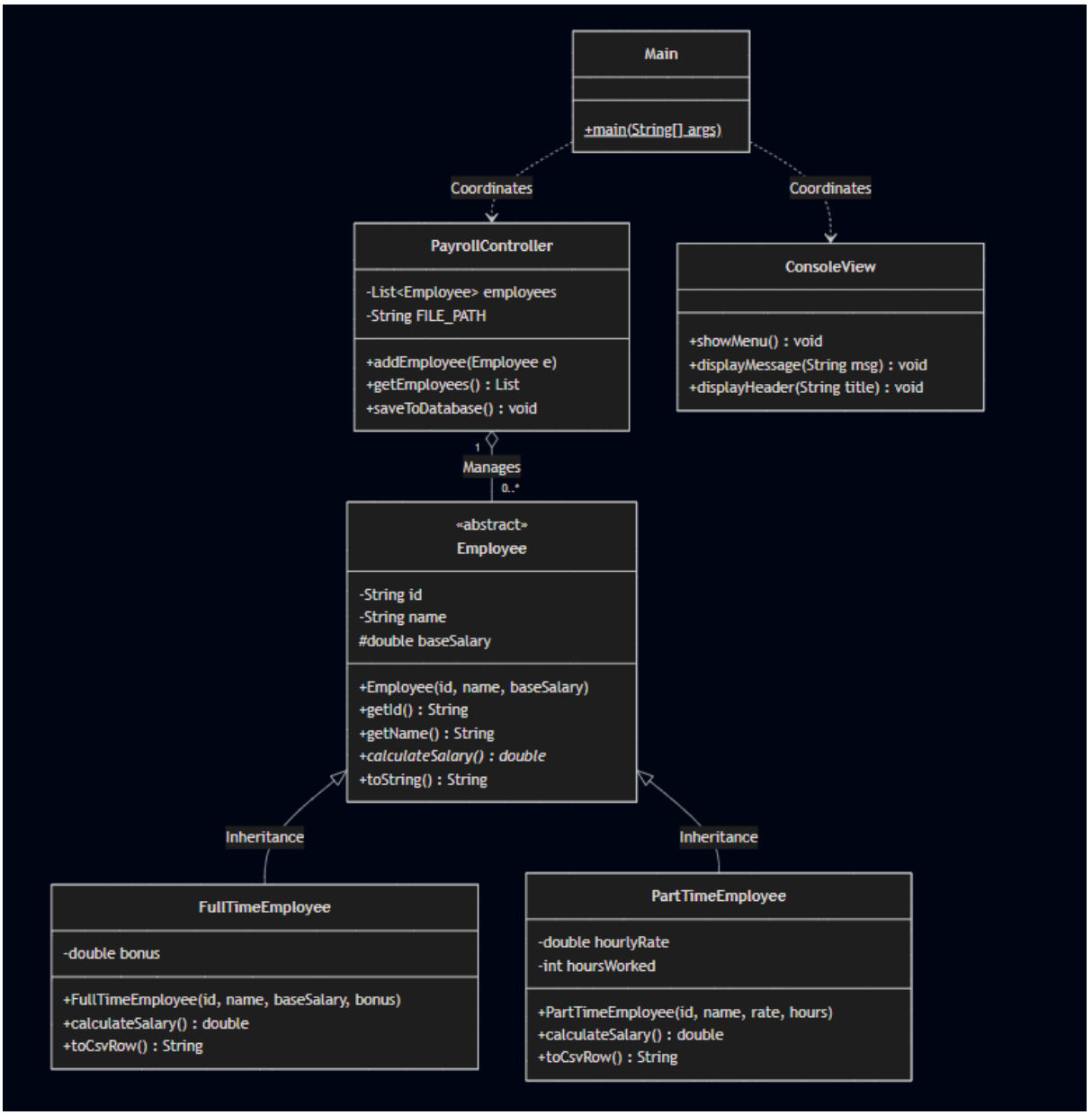
---

3. OOPs Concepts Demonstrated:
   - **Encapsulation:** Used in the *"Employee"* class by making fields like *"id"* and *"name"* private, accessed only via public getters.
   - **Abstraction:** The *"Employee"* class is abstract, hiding the complexity of salary calculations from the user.
   - **Inheritance:** Specialized subclasses (*"FullTimeEmployee", "PartTimeEmployee"*) extend the "Employee" base class.
   - **Polymorphism:** The system calls *"calculateSalary()"* on an *"Employee"* reference, and Java automatically executes the correct version based on the actual object type.
   - **Exception Handling:** Used to manage invalid user inputs (e.g., non-numeric salary) and file errors to prevent crashes.

---

4. System Design:

4.1) UML Class Design (Description)(With mermaid.js editor live):



4.2) Description of major classes:

- **Employee Class:** Represents an individual worker with ID, Name, and an abstract *"calculateSalary()"* method.
- **PayrollController:** Uses an *"ArrayList<Employee>"* to manage records and handles data persistence using *"FileWriter"*.
- **ConsoleView:** Provides a menu-driven interface for adding and viewing employees.

4.3) Package Structure:

```
PayrollManager/
├── src/
│   ├── model/
│   │   ├── Employee.java
│   │   ├── FullTimeEmployee.java
│   │   └── PartTimeEmployee.java
│   ├── controller/
│   │   └── PayrollController.java
│   ├── view/
│   │   └── ConsoleView.java
│   └── Main.java
└── data/
    └── employees.csv
```

4.4) Interaction Flow:

- **User Selection:** The user interacts with the *"ConsoleView"* menu and selects an option (e.g., 'Add Full-Time Employee' or 'View All').
- **Input Capture:** *"Main.java"* uses a *"Scanner"* to read the user's input and specific employee details (ID, Name, Salary/Rate).
- **Request Delegation:** The *"Main"* class sends the captured data to the *"PayrollController"*.
- **Object Creation:** The *"PayrollController"* instantiates the appropriate subclass (*"FullTimeEmployee"* or *"PartTimeEmployee"*) and adds it to the internal *"ArrayList<Employee>"*.
- **Polymorphic Execution:** When 'View All' is selected, the *"PayrollController"* iterates through the list, and the JVM automatically calls the correct *"calculateSalary()"* method for each object type.
- **Data Persistence:** When the "Save" option is selected, the *"PayrollController"* writes the current list of objects into *"data/employees.csv"* using *"FileWriter"*.
- **User Feedback:** The *"ConsoleView"* displays a confirmation message or the calculated payroll results back to the user.

5. Implementation Highlights:

Snippet: Polymorphic Salary Calculation:

```
// Polymorphism in action
for (Employee e : employees) {
    System.out.println(e.getName() + " Salary: " + e.calculateSalary());
}
```

- **Justification:** Allows the system to handle different employee types in a single list without knowing their specific class.

---

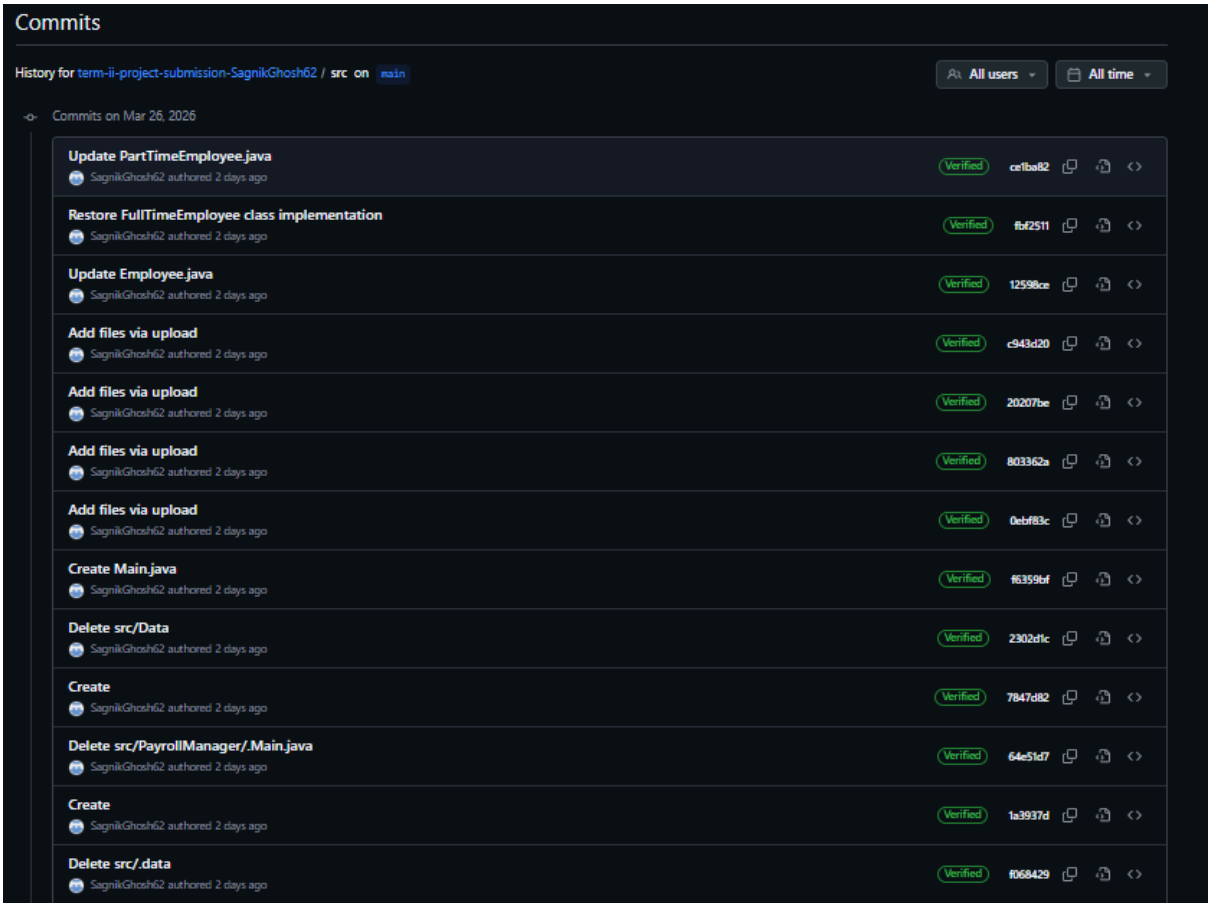
6. Testing & Error Handling:

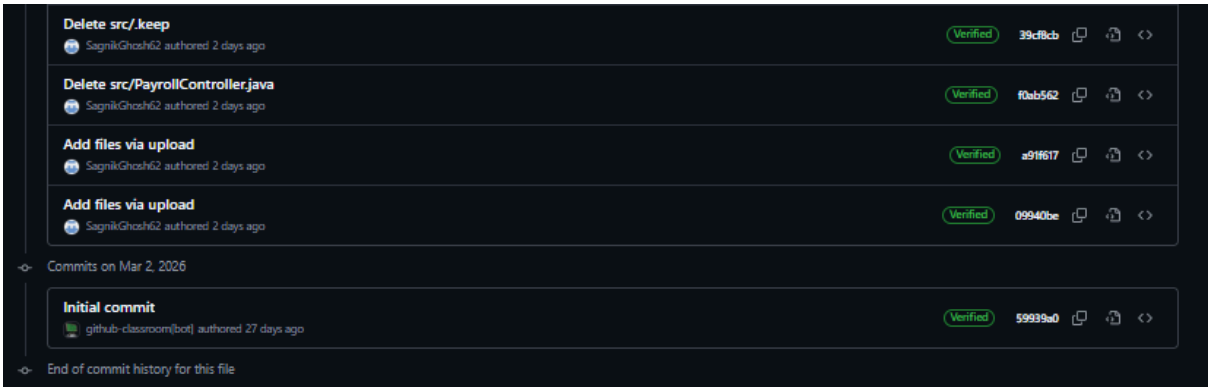
Case	**Expected Result**
Add Full-Time Employee	Record added with Base + Bonus
Add Part-Time Employee	Record added with Hourly Rate * Hours
Invalid Input (Salary)	"Invalid input" message shown via try-catch
File Save/Load	Data persists in *"`employees.csv`"*

---

7. Git Workflow:

- *Strategy*: Frequent commits documenting progression from project initialization to final exception handling.
- *Development Progression*:
    1. Project initialization with MVC folders.
    2. Creation of *"Employee"* hierarchy.
    3. *"PayrollController"* logic and CSV persistence.
    4. Menu system and robust input validation.
- *Commit History*:





---

8. Conclusion & Future Scope:

The **Payroll Manager** successfully applies Java OOP principles to solve real-world organizational needs.

Future Scope:

- Integration of a Database (SQLite via JDBC).
- Development of a GUI using JavaFX.
- Tax calculation and audit logging features.

APPENDIX:

Project Title: Multi-Tiered Employee Payroll Manager

Problem Statement: Small to medium enterprises often struggle with manual payroll processing, leading to human errors, inconsistent tax calculations, and difficulties managing different employee types. This project develops a Java-based application that automates salary computation using tier-specific fiscal logic, with exception handling and persistent data management to ensure accuracy.

Target User: The primary users are “Human Resource (HR) Managers” and “Payroll Administrators” in small organizations who require a reliable, automated tool to manage employee records and monthly salary disbursements.

Core Features:

- Role-Based Salary Calculation
- Employee Record Management

- Tax & Deduction Logic
- Persistent Storage
- Error Reporting

OOPs Concepts used:

- Abstraction: Implementing an abstract 'Employee' class to define essential payroll behaviors while hiding the internal logic of complex tax calculations.
- Inheritance: Extending the base 'Employee' class into specialized subclasses such as 'SalariedEmployee', 'HourlyEmployee', and 'Contractor'.
- Polymorphism: Using a common 'calculatePay()' method that behaves differently across subclasses to handle varying payment structures.
- Exception Handling: Utilizing 'try-catch' blocks and custom exception classes (e.g.,'InvalidHoursException') to manage runtime input errors.
- Collections: Using 'ArrayList' or 'HashMap' to efficiently store, retrieve, and manipulate employee objects.

Proposed Architecture Description: The project will follow a modular N-tier architecture to ensure clean code practices. The Model layer will contain the 'Employee' hierarchy and data encapsulated with private fields and public getters/setters. The Controller layer will manage the business logic for payroll processing and file interactions. Finally, a Console-based View layer will provide a menu-driven interface for user interaction. This separation ensures that the system is easily maintainable and extendable for future updates, such as adding a graphical user interface.

PCCCS495 Proposal Review – APPROVED_MINOR

Project: Multi-Tiered Employee Payroll Manager

Decision: APPROVED_MINOR

Score: 90

Strengths:

The proposal effectively identifies a practical problem (manual payroll processing) that is well-suited for a Java OOP application. It clearly outlines the target users and a comprehensive set of features. A major strength is the strong alignment between the problem domain (different employee types, roles, and fiscal logic) and the proposed OOP concepts (Abstraction, Inheritance, Polymorphism, Exception Handling, Collections), ensuring a solid foundation for applying these principles. The described N-tier architecture (Model-Controller-View) demonstrates a good

understanding of separation of concerns, maintainability, and extensibility, including a foresight for future GUI integration. The explicit mention of encapsulation (private fields, public getters/setters) further reinforces good OOP practices.

Improvements:

While the proposal is strong, a few minor enhancements could be considered: 1) Specify the chosen method for 'Persistent Storage' (e.g., flat file like CSV/JSON, Java Serialization, or a simple embedded database like SQLite via JDBC). 2) Briefly elaborate on the types of errors 'Error Reporting' will cover (e.g., invalid input, file read/write errors, calculation exceptions). 3) For a robust solution, mentioning a plan for unit testing or integration testing would be beneficial, even if not a core requirement for a foundational OOP course.